

Reporte Tarea 2 Bases de Datos

Grupo 33

Integrantes:

Emma Brunetti Fantuzzi - 23645083

Antonia Belén Ríos Donoso - 23644524

Martín Concha Ortega - 24643130

Paso 1 - Línea base y análisis de workload

Para obtener la línea base, medimos cuánto tarda el workload antes de hacer cualquier optimización. Para esto, corremos el workload 2 veces y guardamos la segunda corrida como baseline. En Anexo, se pueden ver los resultados de la terminal (Figura 1). Nuestra baseline de tiempo total de la carga es de **2537 ms** (24,47% más rápida que con caché frío).

Análisis de workload

Se identificaron 11 tipos de consultas que usaban las mismas tablas y eran prácticamente iguales estructuralmente, pero con distintos parámetros de búsqueda. Para visualizarlo mejor, se armó una tabla resumen con el análisis de las 100 consultas (ver Anexo Tabla 1).

Notamos que el tipo de consulta que más se repite es la Q2: usuario por email exacto (case sensitive), la cual representa un 36% del total de las consultas en el workload. Cabe mencionar que Q2 tiene un índice implícito por la restricción UNIQUE en email. Esto podría implicar un tiempo de ejecución menor al analizar los resultados de times_baseline.csv. Los tipos de consulta que son más importantes a la hora de optimizar son aquellas que aparecen con mayor frecuencia en el workload o tienen un alto costo. Por ejemplo, la consulta Q10: Usuarios que reseñaron libros que también compraron, si bien aparece solo 2 veces, es de las más costosas, ya que hace EXISTS cruzando reviews (5.3M tuplas) con orders (2.4M tuplas) y order_items (1M tuplas). Para cada fila de review tiene que verificar si existe una orden correspondiente.

Análisis times_baseline.csv

Este archivo contiene el tiempo de carga de cada consulta. Analizando los resultados (ver Tabla 2 en Anexo), notamos que Q10 consume más de la mitad del workload (53,1%) con tan solo 2 queries, mientras que Q2, con 36 queries, consume menos del 1% (índice implícito mencionado anteriormente). A partir de esto, identificamos 4 clases de consultas que debemos atacar para reducir el tiempo de ejecución significativamente.

Clase consulta	N° queries	Tiempo total (ms)	Porcentaje del workload (total=2537 ms)
Q10: Semi-join reviews + orders	2	1347,3	53,1%
Q3: Revenue por libro en rango de fechas	2	367,5	14,4%
Q6: reviews de un libro ordenadas por fecha	15	231,7	9,1%
Q8: usuario por email (case-insensitive)	5	93,3	3,7%

Paso 2 - Índices

Los índices B+ tree permiten evitar el Seq Scan completo sobre una tabla navegando directamente a las filas relevantes. Su costo es logarítmico en profundidad versus lineal en páginas para un Seq Scan. Se proponen 3 índices:

Índice 1 - idx_orders_date

```
CREATE INDEX idx_orders_date ON orders(order_date);
```

Clase de consulta que ataca → Cubre 3 clases del workload que filtran por rango de order_date sobre 2.400.000 filas; primero, Q3 (revenue por libro, 14,4%), Q5 (órdenes de usuarios premium, 6,7%) y Q11 (órdenes simples por fecha, 3%). Estas representan el 24,1% del workload. Sin índice, las tres ejecutan Parallel Seq Scan completo sobre orders. Las hojas del B+ Tree están encadenadas entre sí en orden,

lo que hace que los range scans sean eficientes, pues el motor navega al inicio del rango y recorre las hojas encadenadas hasta el final sin leer páginas innecesarias. Un solo índice beneficia a las tres clases simultáneamente.

Evidencia experimental:

	<i>Sin índice</i>	<i>Con índice</i>
<i>Plan</i>	Parallel Seq Scan + Sort	Index Scan Backward
<i>Páginas leídas</i>	19.592	53
<i>Tiempo de ejecución</i>	348,499 ms	0,917 ms
<i>Speedup</i>	—	~380x

Plan Anterior:

Parallel Seq Scan on orders (cost=0.00..34592.00 rows=17040 width=22) (actual time=0.162..70.736 rows=14078.00 loops=3)
 Filter: ((order_date >= '2022-09-01 00:00:00'::timestamp without time zone) AND (order_date < '2022-10-01 00:00:00'::timestamp without time zone))
 Rows Removed by Filter: 785922
 Buffers: shared read=19592 written=138
Execution Time: 348.499 ms

Plan Después:

Index Scan Backward using idx_orders_date on orders (cost=0.43..79562.02 rows=40897 width=22) (actual time=0.113..0.896 rows=50.00 loops=1)
 Index Cond: ((order_date >= '2022-09-01 00:00:00'::timestamp without time zone) AND (order_date < '2022-10-01 00:00:00'::timestamp without time zone))
 Index Searches: 1
 Buffers: shared hit=1 read=52
Execution Time: 0.917 ms

Índice 2 - idx_reviews_book_date

CREATE INDEX idx_reviews_book_date ON reviews(book_id, review_date DESC);

Clase de consulta que ataca → 15 instancias del workload (consultas 014, 022, 031, 033, 036, 041, 052, 054, 055, 070, 074, 075, 091, 095, 099) con el patrón WHERE book_id = X ORDER BY review_date DESC LIMIT 20 sobre la tabla reviews (1.000.000 de filas). Sin índice, el optimizador ejecuta un Parallel Seq Scan que recorre toda la tabla para encontrar reseñas de un libro específico, descartando en cada worker ~333.000 filas que no corresponden al book_id buscado. Además, una vez filtradas las filas, se requiere un paso de ordenamiento (sort) por review_date DESC antes de aplicar el LIMIT 20. Se propone un índice compuesto (book_id, review_date DESC) con book_id primero porque es la condición de igualdad, permitiendo al B+ Tree navegar directamente a las hijas del libro buscado, y review_date DESC segundo para que las hojas del índice ya estén ordenadas por fecha descendente, lo que permite satisfacer el ORDER BY review_date DESC LIMIT 20 sin ningún paso de sort adicional.

Evidencia experimental:

	<i>Sin índice</i>	<i>Con índice</i>
<i>Plan</i>	Parallel Seq Scan + Sort	Index Scan
<i>Páginas leídas</i>	12.952	23

Tiempo de ejecucion	314,495 ms	0,088 ms
Speedup	—	~3.573x

Plan Anterior:

Parallel Seq Scan on reviews (cost=0.00..18160.33 rows=41 width=84) (actual time=0.341..28.521 rows=172.33 loops=3)
 Filter: (book_id = 589)
 Rows Removed by Filter: 333.161
 Buffers: shared hit=12.786 read=166
Execution Time: 314.495 ms

Plan Después:

Index Scan using idx_reviews_book_date on reviews (cost=0.42..398.13 rows=98 width=84) (actual time=0.055..0.072 rows=20.00 loops=1)
 Index Cond: (book_id = 589)
 Index Searches: 1
 Buffers: shared hit=20 read=3
Execution Time: 0.088 ms

Índice 3 - idx_users_lower_email

```
CREATE INDEX idx_users_lower_email ON users(LOWER(email));
```

Ataca clase Q8 (consultas 032, 050, 057, 058, 097) con el patrón WHERE LOWER(email) = LOWER('...') sobre la tabla users.

La tabla users ya tiene un índice implícito en email por la constraint UNIQUE, pero ese índice no puede usarse cuando la condición aplica una función LOWER('...'), entonces obliga al optimizador a evaluar LOWER() sobre cada fila, resultando en un Seq Scan completo de la tabla. Se propone un índice que almacena directamente el valor de LOWER(email) en el B+Tree, así al optimizador puede resolver la condición con un Index Scan en O(log n) para no evaluar en cada fila.

Plan Anterior:

Parallel Seq Scan on users
 Filter: (lower(email) = 'zara.zhang71379@mail.com')
 Rows Removed by Filter: 100.000
 Buffers: shared hit=1800
Execution Time: 51.647 ms

Plan Después:

Bitmap Heap Scan on users
 Heap Blocks: exact=1
 Buffers: shared hit=1 read=3
 -> Bitmap Index Scan on idx_users_lower_email
 Index Cond: (lower(email) = 'zara.zhang71379@mail.com')
Execution Time: 0.140 ms

Evidencia experimental:

	Sin índice	Con índice
Plan	Parallel Seq Scan + Filter	Bitmap Index Scan

<i>Páginas leídas</i>	1.800	4
<i>Tiempo de ejecución</i>	51,6 ms	0,14 ms
<i>Speedup</i>	—	~369x

Paso 3 - Materialized view

La clase Q10 corresponde a las consultas 047 y 082, que representan el 53% del tiempo total del workload. Ambas con el patrón:

```
SELECT DISTINCT r.user_id, r.book_id
FROM reviews r
WHERE EXISTS (
  SELECT 1 FROM orders o
  JOIN order_items oi ON oi.order_id = o.order_id
  WHERE o.user_id = r.user_id AND oi.book_id = r.book_id
)
AND r.review_date >= '<fecha_inicio>' AND r.review_date < '<fecha_termino>';
```

El plan sin índices ejecuta un Parallel Hash Semi Join que hace Seq Scan de las 5.3M filas de order_items y 2.4M de orders para construir un hash gigante de todas las compras históricas. Después cruza ese hash con un Seq Scan de reviews filtrado por fecha y finalmente usa 44.088 páginas de disco temporal porque el hash no cabe en la RAM.

Probando índices, el optimizador eligió caminos más lentos que el original: hash enorme o nested loop join con miles de lookups. El problema es verificar el EXISTS ya que requiere cruzar tres tablas grandes sin importar qué índice se use. Se probaron específicamente reviews(review_date, book_id, user_id) y order_items(book_id, order_id), ambos resultaron en tiempos peores que sin índice. Por eso, se propone una matview que precalcula todos los pares (user_id, book_id, review_date) donde el usuario también compró ese libro. La consulta después solo filtra por fecha sobre esa tabla.

```
CREATE MATERIALIZED VIEW reviews_with_purchase AS
SELECT DISTINCT r.user_id, r.book_id, r.review_date
FROM reviews r
WHERE EXISTS (
  SELECT 1 FROM orders o
  JOIN order_items oi ON oi.order_id = o.order_id
  WHERE o.user_id = r.user_id AND oi.book_id = r.book_id
);

CREATE INDEX idx_rwp_date ON reviews_with_purchase(review_date);
```

Las queries 047, 082 se reescriben en workload_after.sql de la siguiente forma:

SELECT DISTINCT user_id, book_id FROM reviews_with_purchase WHERE review_date >= '2022-11-01' AND review_date < '2022-12-01';	SELECT DISTINCT user_id, book_id FROM reviews_with_purchase WHERE review_date >= '2023-04-01' AND review_date < '2023-05-01';
--	--

Con eso, verificamos que ambas devuelven exactamente los mismos resultados que las originales (48 y 99 filas, respectivamente).

	<i>Sin matview</i>	<i>Con matview</i>
--	--------------------	--------------------

Plan	Parallel Hash Semi Join	Bitmap Index Scan
Páginas leídas	52.860	53
Tiempo de ejecución	1618 ms	0,67 ms
Speedup	—	~2415x

La matview queda congelada al momento de crearse. Si los datos de reviews, orders u order_items cambian, habría que ejecutar `REFRESH MATERIALIZED VIEW reviews_with_purchase` para actualizarla. En este caso el costo no aplica (los datos son estáticos). Ver Figuras 2 y 3 del Anexo.

Total cambios: 3 índices nuevos y un matview con su índice.

```
antoniariosdonoso@Antonias-MacBook-Pro-2 Tarea-2-Bases-de-Datos % psql -d bookstore_g33 -c "\di"
List of relations
Schema | Name | Type | Owner | Table
-----|-----|-----|-----|-----
public | authors_pkey | index | antoniariosdonoso | authors
public | books_pkey | index | antoniariosdonoso | books
public | idx_orders_date | index | antoniariosdonoso | orders
public | idx_reviews_book_date | index | antoniariosdonoso | reviews
public | idx_rwp_date | index | antoniariosdonoso | reviews_with_purchase
public | inventory_pkey | index | antoniariosdonoso | inventory
public | order_items_pkey | index | antoniariosdonoso | order_items
public | orders_pkey | index | antoniariosdonoso | orders
public | reviews_pkey | index | antoniariosdonoso | reviews
public | users_email_key | index | antoniariosdonoso | users
public | users_pkey | index | antoniariosdonoso | users
(11 rows)
```

Paso 4 - Mediciones finales

Para validar el impacto real de las optimizaciones propuestas, se levantó una base de datos limpia desde cero utilizando nuevamente `schema.sql` y `load.sql`, evitando contaminar las mediciones con estadísticas o cachés generadas durante la experimentación previa. Sobre esta nueva instancia se aplicaron los índices definidos en `indexes.sql`, la materialized view `reviews_with_purchase` junto a su índice asociado, y posteriormente se ejecutó `ANALYZE` para actualizar las estadísticas del optimizador. Luego se ejecutó nuevamente el workload dos veces utilizando `workload_after.sql`, utilizando la segunda corrida como medición final; a continuación se muestra la comparación entre los tiempos baseline y after para cada clase de consulta.

Clase consulta	N° queries	Baseline (ms)	After (ms)	Speedup	Análisis
Q1: Mostrar libros por género, precio y año	12	46,7	72,35	0.65x	No se creó índice compuesto sobre books, por lo que las consultas siguen realizando filtrado y ordenamiento sobre la tabla, debido a que el impacto sobre el workload total es bajo.
Q2: Usuario mail exacto	36	23.2	10,94	2,12x	Se observa una leve mejora, probablemente asociada a efectos de caché y a una menor presión general sobre el sistema tras las optimizaciones aplicadas.
Q3: Revenue por libro en rango de fechas	2	367,5	794,4	0,46x	El índice <code>orders(order_date)</code> reduce parcialmente el filtrado temporal, pero el costo dominante sigue estando en los joins y agregaciones sobre <code>order_items</code> .
Q4: UPDATE stock inventario	15	7.6	15,06	0,50x	Las actualizaciones empeoraron levemente debido al costo adicional de mantención de índices. Aun así,

					representan una fracción marginal del workload total.
Q5: órdenes de usuarios premium en rango de fechas	4	170,8	218,76	0,78x	El índice orders(order_date) ayudó a reducir el filtrado por rango temporal, pero el join con users sigue siendo costoso.
Q6: reviews de un libro ordenadas por fecha	15	231,7	9,55	24,26x	El índice compuesto (book_id, review_date DESC) permite filtrar y retornar los resultados ya ordenados, eliminando sorting adicional, lo que se traduce en una gran reducción del tiempo de ejecución.
Q7: Conteo libros con ≥ 10 reseñas y rating promedio > 4.0	3	161,181	525,38	0,31x	Estas consultas no fueron optimizadas directamente; estas consultas no fueron optimizadas y empeoraron de 161 a 525 ms. El índice idx_reviews_book_date lleva al planificador a elegir un plan subóptimo para la agregación completa sobre reviews, aumentando el costo en vez de bajarlo.
Q8: usuario por email (case-insensitive)	5	93,3	1,71	54,56x	Excelente mejora gracias al índice funcional users(LOWER(email)), que evita realizar LOWER() sobre toda la tabla en cada consulta.
Q9: Libros por prefijo título (LIKE)	4	7,4	10,54	0,70x	Utiliza scans y sorting simples, los cuales tienen un impacto bajo en el workload, por lo que no se creó índice sobre title.
Q10: Semi-join reviews+orders	2	1347,3	4,48	300,74x	La materialized view reviews_with_purchase precalcula los pares (user_id, book_id) válidos, eliminando el semi-join costoso sobre tablas grandes y el índice sobre review_date acelera aún más el filtrado temporal.
Q11: Orders en rango de fechas	2	76,6	1,06	72,26x	El índice orders(order_date) permitió acceder directamente al rango temporal requerido y aprovechar el LIMIT, reduciendo drásticamente el tiempo de ejecución.

En general, el workload completo pasó de 2537 ms a aproximadamente 1664 ms, obteniendo un speedup cercano a 1,52x, gracias a una reducción de tiempo de las consultas objetivo del análisis inicial (Q6, Q8, Q10 y Q11), principalmente a través de la eliminación de operaciones costosas de semi-join y sorting. En particular, Q10 pasó de representar más de la mitad del tiempo total del workload a ser prácticamente despreciable gracias al uso de la materialized view. Por lo tanto, el efecto global fue positivo y logró disminuir considerablemente el tiempo total de respuesta del sistema.

Anexos

Figura 1: Resultados tras correr workload.sql con caché frío y caliente.

```

antoniariosdonoso@Antonias-MacBook-Pro-2 Tarea-2-Bases-de-Datos % python3 run_workload.py --db bookstore_g33
Connected to bookstore_g33. Running 100 queries from workload.sql.

>>> T_workload = 3351 ms <<<

[antoniariosdonoso@Antonias-MacBook-Pro-2 Tarea-2-Bases-de-Datos % python3 run_workload.py --db bookstore_g33 --csv times_baseline.csv
Connected to bookstore_g33. Running 100 queries from workload.sql.

>>> T_workload = 2537 ms <<<

[Wrote per-query timings to times_baseline.csv
antoniariosdonoso@Antonias-MacBook-Pro-2 Tarea-2-Bases-de-Datos % █

```

Tabla 1: Tabla resumen tras analizar workload completo.

# Query	Qué hace?	Tablas involucradas	Columnas en WHERE, JOIN	N° Queries	Estructura
Q1	Mostrar libros por género, precio y año	books	genre price publication_year	001, 020, 026, 030, 063, 066, 077, 081, 083, 087, 089, 096	SELECT book_id, title, price, publication_year FROM books WHERE genre = 'XXX' AND price BETWEEN XX AND XX AND publication_year >= XXXX ORDER BY publication_year DESC, price ASC LIMIT 50;
Q2	usuario por email exacto	users	email	002, 004, 007, 008, 009, 012, 013, 015, 016, 018, 023, 025, 028, 029, 034, 037, 040, 043, 045, 048, 049, 056, 059, 060, 061, 064, 068, 078, 080, 084, 085, 086, 090, 092, 093, 094	SELECT user_id, is_premium, country FROM users WHERE email = 'XXX';
Q3	Revenue por libro en rango de fechas	orders order_items books	order_date order_id book_id	003, 044	SELECT b.book_id, b.title, SUM(oi.quantity * oi.unit_price) AS revenue FROM orders o JOIN order_items oi ON oi.order_id = o.order_id JOIN books b ON b.book_id = oi.book_id WHERE o.order_date >= 'XXXX-XX-XX' AND o.order_date < 'XXXX-XX-XX' GROUP BY b.book_id, b.title ORDER BY revenue DESC LIMIT 10;
Q4	UPDATE stock en inventario	inventory	book_id warehouse_id	005, 011, 017, 019, 027, 035, 038, 039, 046, 051, 065, 067, 072, 073, 076	UPDATE inventory SET stock_count = stock_count - 1, last_updated = CURRENT_TIMESTAMP WHERE book_id = XXXXX AND warehouse_id = X;
Q5	órdenes de usuarios premium en rango de fechas	orders users	user_id is_premium order_date	006, 010, 024, 069	SELECT o.order_id, o.user_id, o.order_date, o.total_amount FROM orders o JOIN users u ON u.user_id = o.user_id WHERE u.is_premium = TRUE AND o.order_date >= 'XXXX-XX-XX' AND o.order_date < 'XXXX-XX-XX' ORDER BY o.order_date DESC;
Q6	Reviews de un libro ordenadas por fecha	reviews	book_id	014, 022, 031, 033, 036, 041, 052, 054, 055, 070, 074, 075, 091, 095, 099	SELECT review_id, user_id, rating, review_text, review_date FROM reviews WHERE book_id = XXX ORDER BY review_date DESC LIMIT 20;
Q7	Conteo libros con ≥10 reseñas y rating promedio > 4.0	reviews	GROUP BY book_id, HAVING COUNT() AVG(rating)	021, 088, 100	SELECT COUNT(*) FROM (SELECT book_id FROM reviews GROUP BY book_id HAVING COUNT(*) >= 10 AND AVG(rating) > 4.0) sub;
Q8	usuario por email (case-insensitive)	users	email	032, 050, 057, 058, 097	SELECT user_id, is_premium FROM users WHERE LOWER(email) = LOWER('ZARA.ZHANG71379@MAIL.COM');
Q9	Búsqueda libros por prefijo de título (LIKE 'Xx%')	books	title	042, 053, 071, 098	SELECT book_id, title, price FROM books WHERE title LIKE 'Xx%' ORDER BY title LIMIT 25;
Q10	Usuarios que reseñaron libros que también compraron en rango de fechas	reviews orders	Order_items Order_id User_id book_id	047, 082	SELECT DISTINCT r.user_id, r.book_id FROM reviews r WHERE EXISTS (SELECT 1 FROM orders o JOIN order_items oi ON oi.order_id = o.order_id WHERE o.user_id = r.user_id AND oi.book_id = r.book_id) AND r.review_date >= 'XXXX-XX-XX' AND r.review_date < 'XXXX-XX-XX';
Q11	Orders en rango de	orders	order_date	062, 079	SELECT order_id, user_id, order_date, total_amount

	fechas			FROM orders WHERE order_date >= 'XXXX-XX-XX' AND order_date < 'XXXX-XX-XX' ORDER BY order_date DESC LIMIT 50;
--	--------	--	--	---

Tabla 2: Análisis times_baseline.csv.

Clase consulta	N° queries	Tiempo total (ms)	Porcentaje del workload (total=2537 ms)
Q10: Semi-join reviews+orders	2	1347,3	53,1%
Q3: Revenue por libro en rango de fechas	2	367,5	14,4%
Q6: reviews de un libro ordenadas por fecha	15	231,7	9,1%
Q5: órdenes de usuarios premium en rango de fechas	4	170,8	6,7%
Q7: Conteo libros con ≥10 reseñas y rating promedio > 4.0	3	161,181	6,3%
Q8: usuario por email (case-insensitive)	5	93,3	3,7%
Q11: Orders en rango de fechas	2	76,6	3%
Q1: Mostrar libros por género, precio y año	12	46,7	1,8%
Q2: Usuario mail exacto	36	23,2	0,9%
Q4: UPDATE stock inventario	15	7,6	0.3%
Q9: Libros por prefijo título (LIKE)	4	7,4	0.3%

Figura 2: EXPLAIN ANALYZE antes del Matview

```
QUERY PLAN
-----
Unique (cost=280131.68..280148.32 rows=134 width=8) (actual time=1596.852..1618.537 rows=48 loops=1)
  Buffers: shared hit=13712 read=52868, temp read=44088 written=44944
  -> Gather Merge (cost=280131.68..280147.65 rows=134 width=8) (actual time=1596.852..1618.532 rows=48 loops=1)
    Workers Planned: 2
    Workers Launched: 2
    Buffers: shared hit=13712 read=52868, temp read=44088 written=44944
    -> Unique (cost=199131.66..199132.16 rows=67 width=8) (actual time=1593.443..1593.449 rows=16 loops=3)
      Buffers: shared hit=13712 read=52868, temp read=44088 written=44944
      -> Sort (cost=199131.66..199131.82 rows=67 width=8) (actual time=1593.442..1593.447 rows=16 loops=3)
        Sort Key: r.user_id, r.book_id
        Sort Method: quicksort Memory: 25kB
        Buffers: shared hit=13712 read=52868, temp read=44088 written=44944
        Worker 0: Sort Method: quicksort Memory: 25kB
        Worker 1: Sort Method: quicksort Memory: 25kB
        -> Parallel Hash Semi Join (cost=171840.84..199129.62 rows=67 width=8) (actual time=1535.294..1593.391 rows=16 loops=3)
          Hash Cond: (r.user_id = o.user_id) AND (r.book_id = oi.book_id)
          Buffers: shared hit=13640 read=52868, temp read=44088 written=44944
          -> Parallel Seq Scan on reviews r (cost=0.00..19202.00 rows=67 width=8) (actual time=0.669..230.396 rows=181 loops=3)
            Filter: ((review_date >= '2022-11-01 00:00:00'::timestamp without time zone) AND (review_date < '2022-12-01 00:00:00'::timestamp without time zone))
            Rows Removed by Filter: 333233
            Buffers: shared hit=178 read=12774
          -> Parallel Hash (cost=129123.10..129123.10 rows=221716 width=8) (actual time=1288.758..1288.752 rows=1773693 loops=3)
            Buckets: 262144 Batches: 64 Memory Usage: 537648
            Buffers: shared hit=13618 read=40886, temp read=25934 written=46312
            -> Parallel Hash Join (cost=45999.00..129123.10 rows=221716 width=8) (actual time=938.841..1132.553 rows=1773693 loops=3)
              Hash Cond: (oi.order_id = o.order_id)
              Buffers: shared hit=13618 read=40886, temp read=25934 written=26872
              -> Parallel Seq Scan on order_items oi (cost=0.00..56875.16 rows=221716 width=8) (actual time=1.068..534.511 rows=1773693 loops=3)
                Buffers: shared hit=9 read=33886
              -> Parallel Hash (cost=29592.00..29592.00 rows=1000000 width=8) (actual time=256.766..256.768 rows=800000 loops=3)
                Buckets: 262144 Batches: 16 Memory Usage: 796898
                Buffers: shared hit=13314 read=6276, temp written=7788
                -> Parallel Seq Scan on orders o (cost=0.00..29592.00 rows=1000000 width=8) (actual time=0.281..149.749 rows=800000 loops=3)
                  Buffers: shared hit=13314 read=6278
Planning:
  Buffers: shared hit=283 read=29
Planning Time: 5.113 ms
Execution Time: 1618.638 ms
(38 rows)
```

Figura 3: EXPLAIN ANALYZE después del Matview

```
QUERY PLAN
-----
Unique (cost=76.27..76.42 rows=20 width=8) (actual time=0.412..0.449 rows=48 loops=1)
  Buffers: shared hit=53
  -> Sort (cost=76.27..76.32 rows=20 width=8) (actual time=0.411..0.428 rows=48 loops=1)
    Sort Key: user_id, book_id
    Sort Method: quicksort Memory: 26kB
    Buffers: shared hit=53
    -> Bitmap Heap Scan on reviews_with_purchase (cost=4.50..75.84 rows=20 width=8) (actual time=0.056..0.345 rows=48 loops=1)
      Recheck Cond: ((review_date >= '2022-11-01 00:00:00'::timestamp without time zone) AND (review_date < '2022-12-01 00:00:00'::timestamp without time zone))
      Heap Blocks: exact=48
      Buffers: shared hit=50
      -> Bitmap Index Scan on idx_rwp_date (cost=0.00..4.50 rows=20 width=8) (actual time=0.025..0.026 rows=48 loops=1)
        Index Cond: ((review_date >= '2022-11-01 00:00:00'::timestamp without time zone) AND (review_date < '2022-12-01 00:00:00'::timestamp without time zone))
        Buffers: shared hit=2
Planning:
  Buffers: shared hit=109
Planning Time: 1.062 ms
Execution Time: 0.678 ms
(17 rows)
```